# Ejercicios de aplicación 🤓

*Arquitectura de software*

**L2. Eva Durán**

Es hora de que pongas en práctica todo lo aprendido. 🤓

Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

**¡Manos a la obra!**

1. **Desafío** 🎯

   **Consigna:** Define escenarios de calidad específicos para un sistema de gestión de reservas en línea. Elige atributos de calidad clave, como seguridad, rendimiento y usabilidad, y crea tres escenarios de calidad para cada atributo, especificando estímulo, contexto, respuesta esperada y métrica de respuesta. Luego, diseña un diagrama básico de la arquitectura que soporte estos atributos de calidad.

   **Pasos:**

   1. Selecciona tres atributos de calidad prioritarios para el sistema y describe su relevancia.
   2. Define tres escenarios de calidad para cada atributo, detallando estímulo, contexto, respuesta esperada y métrica.
   3. Crea un diagrama arquitectónico básico, usando herramientas de diseño, que represente cómo el sistema soportará estos escenarios de calidad.

4. **¿Dónde se lleva a cabo?**

   Puedes realizar este desafío en una herramienta de diagramación como **Lucidchart** o **Draw.io** para crear el diseño arquitectónico. Para documentar los atributos de calidad y escenarios, puedes usar **Word**, **Google Docs** o **Notion**.

5. **Tiempo de dedicación** ⏳

   2 Horas.

6. **Recursos** 🛠️

   - **Manual del módulo** para repasar conceptos sobre atributos y

escenarios de calidad.

- Referencia a la **ISO/IEC 25010**, que define los principales atributos de calidad en software, para asegurar una selección fundamentada de atributos.
- Guías en línea de diseño de escenarios de calidad, que ofrecen ejemplos de cómo detallar cada elemento de los escenarios (estímulo, contexto, respuesta esperada y métrica).

7. **Plus ➕**

   Opcional: Para profundizar en el desafío, agrega un análisis de riesgos para cada escenario de calidad, señalando los puntos críticos y proponiendo soluciones de mitigación. Además, puedes investigar cómo marcos de calidad como **AWS Well-Architected Framework** pueden fortalecer la arquitectura para cumplir con estos escenarios de calidad.

## DESARROLLO

Los escenarios de calidad son herramientas clave en la arquitectura de software, permiten definir metas claras, prevenir problemas, medir, evaluar y validar si el sistema cumple con los objetivos de calidad.

Existen diferentes atributos de calidad, entre los principales se encuentran: rendimiento, usabilidad, seguridad, mantenibilidad, disponibilidad, escalabilidad y compatibilidad. Para este trabajo se seleccionaron 3 atributos críticos de calidad: **Seguridad, Rendimiento y Usabilidad**.

La seguridad es fundamental para proteger datos sensibles de usuarios y prevenir accesos no autorizados, ataques maliciosos o pérdida de datos.

El rendimiento es la capacidad del sistema de procesar y responder a las solicitudes en un tiempo adecuado. Permite garantizar una experiencia fluida.

La usabilidad se enfoca en la facilidad con que los usuarios pueden interactuar con el sistema. Busca mejorar la satisfacción del usuario.

# Ejercicios de aplicación 🤓

*Arquitectura de software*

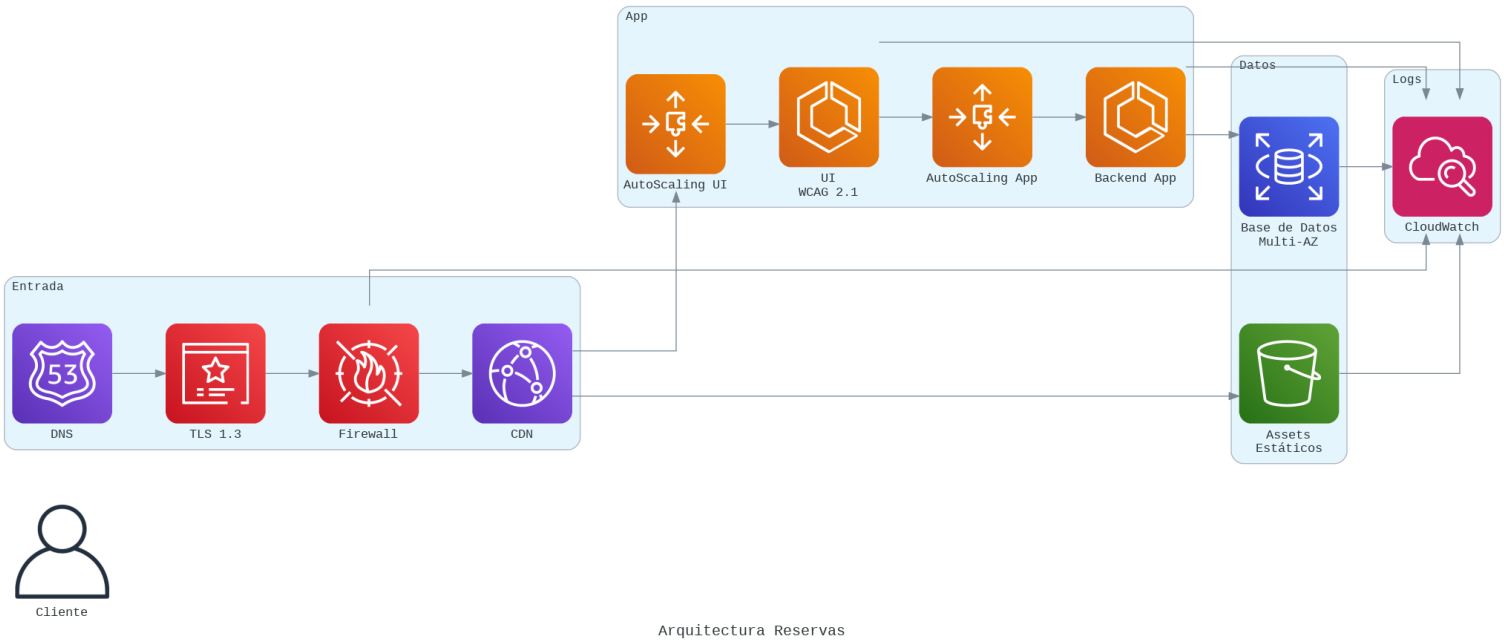
| Atributo | Escenario | Estímulo | Contexto | Respuesta esperada | Métrica de respuesta |
|---|---|---|---|---|---|
| **Seguridad** | Intento de acceso no autorizado | Actor externo sin credenciales de acceso | Durante la operación normal del sistema | El sistema bloquea el acceso e informa al equipo mediante alerta de seguridad | 100% de intentos bloqueados, log registrados en < 5s |
| | Transmisión de datos personales | Envío de datos sensibles (como una transacción) | Formularios/pago | Los datos son encriptados en tránsito y almacenamiento (cuando lleguen al servidor) | Cifrado activo configurado y operando correctamente y sin fugas. |
| | Acceso con roles no autorizados | Usuario autenticado intenta acción restringida (por rol) | Durante una sesión activa de usuario | Accedo denegado y se registra el intento | 0% de accesos fuera de rol; 100% registrados. |
| **Rendimiento** | Aumento de tráfico durante evento especial | Tráfico se duplica en 5 min | Evento promocional | Sistema escala horizontalmente sin afectar la experiencia | Tiempo de respuesta < 3s<br>Disponibilidad >99,9% |
| | Consulta masiva de disponibilidad | 1000 usuarios realizan búsquedas simultáneas | Durante evento especial o planificación de vacaciones | El sistema responde a las consultas sin errores. | Éxito > 95 % en respuestas sin "timeout" |
| | Carga regular en horario punta | 200 usuarios acceden simultáneamente | Horario punta de 18:00 a 20:00 hrs. | El sistema mantiene el tiempo de respuesta estable | Latencia < 2 s promedio |

# Ejercicios de aplicación 🤓

*Arquitectura de software*

| Atributo | Escenario | Estímulo | Contexto | Respuesta esperada | Métrica de respuesta |
|---|---|---|---|---|---|
| **Usabilidad** | Usuario primerizo ingresa al sistema | Nuevo usuario accede sin tutorial | Primera visita al sitio web | La interfaz guía al usuario intuitivamente | Tasa de rebote de nuevos usuarios < 15% |
| | Error al completar una reserva | Usuario comete un error en el formulario | Al completar una reserva en alta demanda | El sistema entrega mensajes claros y sugerencias | Corrección exitosa >90% tras error guiado |
| | Accesibilidad para personas con discapacidad | Usuario con lector de pantalla intenta navegar por el sitio web | Navegación desde notebook con lector NVDA (*NonVisual Desktop Access*) | El sistema es completamente navegable con lector | Cumplimiento WCAG2.1 AA o superior (estándar internacional para accesibilidad web) |

**Diagrama**:




En este caso se presenta un diagrama para un sistema de reservas en línea, diseñada para cubrir los escenarios de calidad mencionados anteriormente.

## Desglose y justificación de la arquitectura

1. Capa de Entrada (seguridad y acceso):
    a. Route 53 (DNS): Resuelve los nombres de dominio del sitio, por ejemplo reservas-ejemplo.com
    b. TLS.13 (ACM): Cifra el tráfico desde el navegador el usuario, usando certificados DDL/TLS (escenario 2 de seguridad)
    c. Firewall (AWS WAF): Protege contra ataques web.
    d. CloudFront (CDN): Distribuye contenido (páginas, imágenes, archivos) globalmente para reducir la latencia.

Esta capa garantiza la seguridad de conexión, protección contra ataques y rendimiento global.

2. Capa de Aplicación (lógica de negocio y usabilidad):
    a. AutoScaling UI: Escala automáticamente las instancias que sirven la interfaz gráfica (frontend).
    b. ECS UI (WCAG 2.1): Contenedor de la interfaz de usuario, cumpliendo los estándares WCAG 2.1 de accesibilidad web.
    c. AutoScaling App: Escala el backend que procesa las reservas y validaciones.
    d. ECS App: Contiene la lógica del sistema de reservas (APIs, reglas de negocio).

Esta capa asegura rendimiento bajo carga, disponibilidad y soporte a la usabilidad (que sea accesible y fluida).

3. Capa de Persistencia (datos):
    a. Base de Datos Multi-AZ (RDS): Base de datos replicada en múltiples zonas para alta disponibilidad y encriptación.
    b. S3: almacenamiento de archivos como imágenes, íconos o documentos.

Esta capa respalda los datos de los usuarios y garantiza que estén seguros y disponibles.

4. Logs y Monitoreo:
    a. CloudWatch: Recibe métricas, logs y alertas de todos los componentes anteriores para:
        i. Medir tiempos de respuesta
        ii. Detectar intentos fallidos
        iii. Ver patrones de uso
        iv. Emitir alertas

Esta capa permite monitorear el sistema y medir las métricas de calidad definidas en la tabla.

## Análisis de Riesgos

| **Escenario** | **Riesgo crítico** | **Mitigación sugerida** |
|---|---|---|
| Acceso no autorizado (Seguridad) | WAF (*Web Application Firewall*) mal configurado o permisos sobreabiertos | Revisar reglas WAF, MFA, auditorías frecuentes. |
| Duplicación de tráfico (Rendimiento) | Latencia al escalar lento o límites de recursos | Pre-provisión mínima*, uso de AWS Auto Scaling con métricas adecuadas. |
| Usuario novato (Usabilidad) | Interfaz confusa o mensajes poco claros | Tests de usabilidad, prototipos, analíticas UX para mejorar iterativamente. |

**La pre-provisión mínima se refiere a tener una cantidad base de recursos ya disponibles, incluso antes de que llegue la carga real.*